

L.J. Institute of Computer Applications



PROJECT REPORT ON

Vote Chain

COURSE NAME: Software Project-1

GROUP NO: 11

SEMESTER: 3

UNDER THE GUIDANCE OF:

DHAVAL SHAH

SUBMITTED BY:

Roll No.	NAME	ENROLLMENT NO.
B-54	RANA DHRUV HARESHKUMAR	24004401110294

ACKNOWLEDGEMENT

It gives us great pleasure in submitting this project entitled “Vote Chain” as a part of the curriculum of MCA (Semester 3).

I avail this opportunity to express my heartfelt gratitude to a number of people who extend their full support and co-operation in developing this project and also imparting knowledge to us in various other domain of software technology.

I would like to take opportunity to thank my college, **L.J School of Computer Application, Ahmedabad**, for giving us this tremendous opportunity to work in real-time project.

I am thankful to my internal project guide **Mr. Dhaval Shah** has been an exceptional teacher and motivator. His inspirational ideas and advices have kept us going during the training period.

I am thankful to our head of director **Dr. Bhavin Shah** for encouraging and faculties of the institute for their constant guidance not only during training period but also throughout college career.

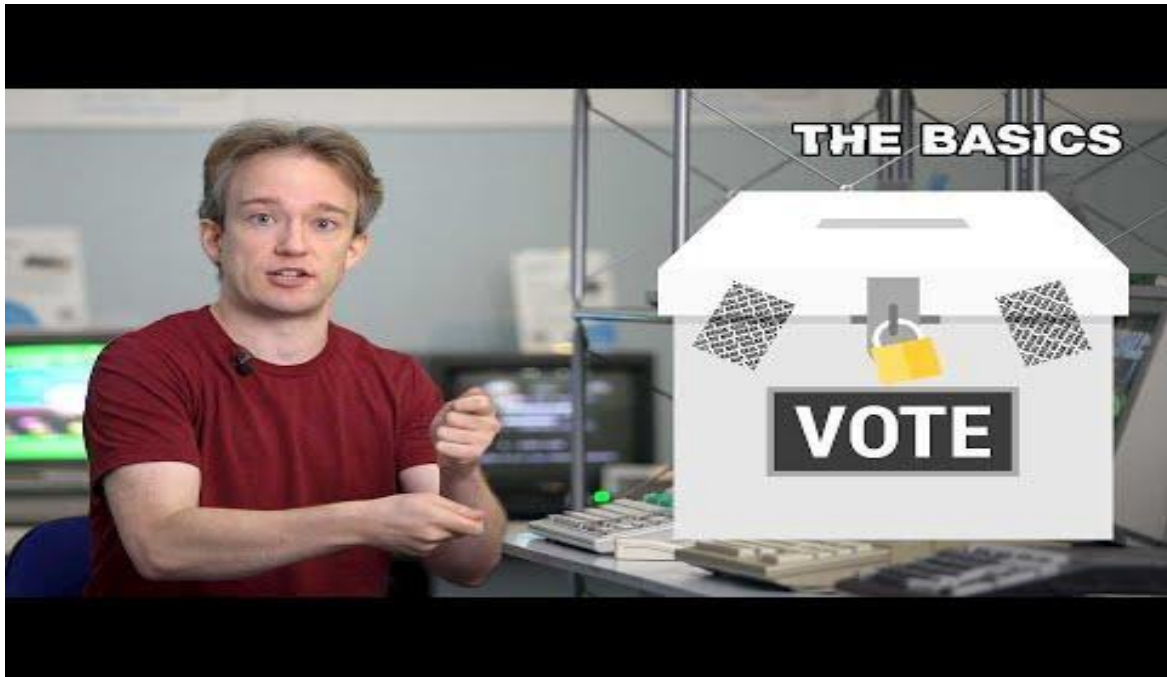
Finally, I own a special debt to our family and friends for their support, blessing and encouragement for me.

INDEX

Sr No.	Content	Page No.
1.	Problem Definition	1
2.	Objective of the New System	2
3.	Core Components	4
4.	Need of the new system	11
5.	Scope of the new system	12
6.	Project Profile	16
7.	Advantages and Limitations of the Proposed System	17
8.	Proposed Time Line Chart	19
9.	Targeted User	21
10.	Requirement Determination	22
11.	Requirement Specification	23
12.	Use Case Diagram	34
13.	Data Dictionary	36
14.	User Interface Design	41
15.	Report Design	61
16.	Coding Standards	62
17.	Test cases	70
18.	Proposed Enhancement	77
19.	Conclusion	80
20.	Bibliography	82

What is the problem with EVM?





Video Link :

<https://youtu.be/LkH2r-sNjQs?si=2Ft5rMrPOWphBmLu>

• Problem Definition:

Traditional voting systems, whether paper-based or centralized electronic systems, often face a range of challenges that hinder efficiency and voter confidence. In conventional paper-based voting, issues such as long queues at polling stations, human errors in manual vote counting, and logistical difficulties in handling large numbers of ballots are common. These inefficiencies not only slow down the voting and result-tallying process but can also discourage voter participation due to inconvenience.

Such systems are vulnerable to various forms of manipulation, including ballot tampering, vote rigging, and unauthorized access to sensitive data. Even electronic voting systems, while more advanced, are frequently centralized—meaning all votes are stored and processed on a single server or under the control of one authority. This centralization introduces potential risks of cyberattacks, system failures, and lack of accountability, as voters have limited visibility into how their votes are recorded and counted.

Consequently, these limitations create distrust among citizens and raise questions about the integrity and fairness of the election process. Therefore, there is a growing need for a **secure, transparent, and reliable voting system** that minimizes human error, prevents fraud, ensures data integrity, and allows voters to verify that their votes have been accurately recorded. Such a system should aim to enhance accessibility, strengthen voter confidence, and ultimately promote higher participation in democratic processes.

• **Objective:**

The primary objective of the **VoteChain system** is to develop a **secure, transparent, and tamper-proof online voting platform** by leveraging the unique features of **blockchain technology**. The system aims to address the longstanding challenges of traditional and centralized electronic voting systems—such as lack of transparency, vulnerability to fraud, and limited voter trust—by introducing a decentralized, verifiable, and immutable framework for recording and managing votes.

In this system, each vote is treated as a **transaction** that is securely encrypted and recorded on a distributed ledger. This ensures that once a vote is cast, it cannot be altered, deleted, or manipulated by any single party, thereby preserving the **integrity and authenticity** of the election data. Blockchain's **decentralized architecture** eliminates the need for a central authority to manage or verify votes, reducing the risk of system failures, hacking, or tampering.

Furthermore, the VoteChain platform promotes **transparency and auditability** by allowing all stakeholders—voters, candidates, and election authorities—to verify election results in real time without compromising voter privacy. Each transaction on the blockchain is traceable and timestamped, ensuring that the entire voting process remains open to verification while maintaining the **anonymity and confidentiality** of individual voters.

Ultimately, the VoteChain system seeks to enhance **voter confidence, participation, and democratic integrity** by providing a next-generation digital voting solution that is resistant to fraud, fully transparent, and technologically trustworthy.

- **Core Components:**

- 1. Frontend:**

- **Technologies:** HTML, CSS, Bootstrap 5

- The frontend serves as the **user interface layer** of the system, designed to be responsive, intuitive, and accessible across different devices.

- **Key Features:**

- **Responsive Design:** Built with Bootstrap 5's grid system and components to ensure compatibility with desktops, tablets, and smartphones.
 - **Voter Login Interface:** Allows users to securely log in with unique credentials, integrating with backend authentication APIs.
 - **Candidate Display:** Presents candidate profiles with their names, party details, and manifestos fetched dynamically from the database.
 - **Vote Casting View:** Provides a clear and simple voting interface that ensures users can select and submit their votes easily.
 - **Result Viewing Page:** Displays real-time or post-election results fetched from the backend, showing total votes per candidate and overall election statistics.

2. Backend

- **Technology:** Python (Django Framework)

The backend acts as the **core logic and control layer** of the system, managing user sessions, data flow, and overall election functionality.

- **Responsibilities:**

- **User Management:** Handles voter and admin registration, login, and authentication using Django's built-in user model and cryptographic methods.
- **Election Management:** Controls the entire election process — creation, activation, vote collection, and result computation.
- **System Workflow:** Coordinates interactions between the frontend, blockchain layer, and database to ensure data consistency and security.
- **API Endpoints:** Exposes RESTful APIs for frontend communication (e.g., fetching candidate lists, submitting votes, retrieving results).

3. Database Layer

- **Technology:** SQLite

The database functions as the **data storage and retrieval layer**, ensuring persistent and reliable data management.

- **Data Handled:**

- **Voters:** Stores details such as voter ID, credentials, and participation records.
- **Elections:** Contains election metadata like titles, start/end times, and statuses.
- **Candidates:** Holds candidate information including name, party, biography, and election association.
- **Votes & Results:** Maintains the link between voters and their cast votes, while ensuring anonymity and integrity of voting records.

- **Key Features:**

- Lightweight and efficient, ideal for development or small-scale deployments.
- Uses Django's ORM for easy query management and relationship handling.

4. Blockchain Layer

- **Technology:** Custom Python-based Blockchain

This layer adds a **security and transparency mechanism** to the election process by leveraging blockchain principles.

- **Core Functions:**

- **Immutable Vote Recording:** Each vote is treated as a transaction that is securely recorded in a block.
- **Cryptographic Linking:** Every block contains a SHA-256 hash linking it to the previous block, ensuring the chain's integrity.
- **Tamper Resistance:** Once a block is added to the chain, it cannot be modified without altering all subsequent blocks, preventing data manipulation.
- **Transparency and Verification:** The blockchain ledger can be audited to verify the authenticity of election results.

5. Authentication & Authorization Module

- **Technology:** Django Authentication + Cryptography Libraries
Ensures **secure access control** for both voters and administrators.
- **Features:**
 - **Voter Authentication:** Verifies voter identities through secure login credentials, ensuring each user can vote only once.
 - **Admin Authentication:** Provides role-based access, giving administrative privileges to authorized personnel only.
 - **Encryption:** Uses cryptographic algorithms to protect sensitive data such as passwords .

6. Admin Panel

- **Technology:** Django Admin Interface (Customized)

The admin panel provides a **management dashboard** for administrators to oversee and control the election process.

- **Capabilities:**

- **Election Management:** Add, update, or delete elections and control their active status.
- **Candidate Management:** Manage candidate details, including profiles and election assignments.
- **Real-Time Monitoring:** View ongoing vote counts, voter participation rates, and election statistics.
- **Election Lifecycle Control:** Securely start or end elections to maintain procedural integrity.

7. Voter Module

- **Technology:** Django Views + Frontend Integration

Provides a **personalized interface** for voters to engage with the election process.

- **Functions:**

- **Announcements:** Displays notifications, deadlines, and general election updates.
- **Voting Access:** Allows voters to cast their vote securely during an active election period.
- **Results Viewing:** Enables users to view official election results once voting concludes.
- **User Dashboard:** Offers a simple overview of elections the voter is eligible to participate in.

- **Need of the New System:**

- The current voting mechanisms—both manual and centralized electronic systems—face significant challenges such as security vulnerabilities, vote tampering, lack of transparency, and low voter turnout due to accessibility issues. These limitations compromise the integrity of elections and reduce public trust in the process.
- Vote Chain addresses these issues by integrating blockchain technology and a web-based interface to build a decentralized, tamper-proof, and transparent voting system.

- **Scope of the New System**

The scope of this project encompasses the design, development, and implementation of a **blockchain-integrated application** built using **Python** as the core development language and **SHA-256** as the cryptographic hashing algorithm. The system is tailored specifically for **users in India**, focusing on secure, transparent, and immutable digital record management within a user-friendly and accessible framework.

- 1. Functional Scope**

The system aims to provide a **secure and verifiable environment** for logging and managing sensitive transactions or records. Its blockchain backbone ensures that all entries—whether votes, documents, or data logs—are **permanent, tamper-proof, and traceable**.

Key functional components include:

- **Blockchain Module:**
 - Creation and validation of new blocks using SHA-256 hashing.
 - Secure chaining of blocks to maintain historical data integrity.
- **Application Layer:**
 - User-facing interface for data input, record viewing, and transaction tracking.

- Real-time display of blockchain-verified data logs or outcomes.
- **User Management:**
 - Secure authentication and role-based access control (e.g., admin, user).
 - Session management and identity verification to prevent misuse.
- **Data Storage:**
 - Use of a structured database (SQLite) to store metadata, user profiles, and auxiliary application data.
 - Synchronization between the database and blockchain to ensure data consistency.

2. Geographic and Demographic Scope

This application is designed **specifically for the Indian context**, addressing local challenges and opportunities in digital adoption.

3. Technological Scope

- **Core Technologies:**
 - **Programming Language:** Python (for both blockchain and backend logic).
 - **Hashing Algorithm:** SHA-256 for cryptographic data linkage.
 - **Frameworks:** Django for backend web functionality.
 - **Frontend:** HTML, CSS, Bootstrap 5 for responsive design and accessibility.
 - **Database:** SQLite for efficient, reliable data storage.
- **Security Mechanisms:**
 - End-to-end encryption for data transactions.
 - Blockchain validation for immutability and trust assurance.
 - Role-based authorization to restrict sensitive operations.

4. Operational Scope

The system will function as a **secure, web-based platform** where users can interact, record, and verify data through blockchain integration.

Operational objectives include:

- Ensuring **integrity and non-repudiation** of all transactions.
- Providing **real-time verification** of blockchain entries.
- Delivering a **user-friendly interface** that simplifies blockchain use for non-technical individuals.
- Maintaining **system scalability**, allowing future expansion to additional modules or larger datasets.

5. Limitations (Out-of-Scope Areas)

While the system offers robust blockchain integration and secure functionality, certain areas fall outside its current scope:

- Implementation of advanced consensus algorithms (Proof of Work).
- Mobile-native (Android/iOS) applications — though future versions may include them.
- High-volume transaction in future scalability comparable to industrial-grade blockchain networks (e.g., Ethereum, Hyperledger).

6. Future Scope

The system is designed with extensibility in mind, allowing future upgrades such as:

- Integration with **AI-driven anomaly detection** to identify fraudulent activity.
- Development of **mobile applications** for broader accessibility.
- Transition to **distributed ledger infrastructure** for enhanced decentralization.

• Project Profile

Field	Details
Project Title	Vote Chain – Blockchain-Based Online Voting System
Technology Stack	Frontend: HTML, CSS, Bootstrap 5 Backend: Python (Django) Database: SQLite Blockchain Layer: Custom Python-based Blockchain
Platform	Web Application – Accessible via modern web browsers
Development Tools	Visual Studio Code, Default SQLite DB , Git
Target Users	Voters, Admin
Purpose	To provide a secure, tamper-proof, and transparent online voting experience
Methodology Used	Incremental Model

Advantages and Limitations of the Proposed System

- **Advantages**

1. Enhanced Security: -

- Every vote is recorded as a block in the blockchain, making it tamper-proof and immutable.
- Voter authentication is handled securely using session management and cryptography.
-

2. Transparency and Trust: -

- The blockchain ledger makes vote tracking and verification possible without revealing voter identity.
- Admins can verify the integrity of votes and ensure no vote is altered or deleted.

3. Remote Accessibility: -

- Voters can cast their votes from anywhere, using a web browser.

4. One-Person-One-Vote Enforcement: -

- System prevents multiple votes from the same user using login tracking and vote status checks.

5. User-Friendly Interface: -

- Built using HTML, CSS, and Bootstrap 5, ensuring a responsive and clean UI.
- Easy navigation for both admin and voters

• Limitations

1. Internet Dependency: -

Requires stable internet access, which may exclude some rural or underprivileged users.

2. Digital Literacy Requirement: -

Voters need basic computer or smartphone knowledge to use the system effectively.

3. Initial Setup Complexity: -

Integrating blockchain with web-based platforms can be technically complex during development and deployment.

4. Voter Authentication Challenges: -

Ensuring that only eligible voters participate without breaching privacy or accessibility can be difficult.

5. Hardware & Software Requirements: -

Admins and voters must have access to compatible devices and updated browsers.

- **Proposed Time Line Chart**

Proposed Time Line Chart

	August			September			October			November		
	Week 1	Week 3	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 8	Week 10	Week 11	Wevek 12
Initiation												
Requirement												
Frontend												
Backend												
Integration												
Advanced Features												
Testing												
Deployment												
User Testing												

Week 1	Collect all project needs and requirements
Week 2	Prepare project plan and finalize modules
Week 3	Develop Login & Registration module
Week 4	Create Database Design & Schema
Week 5	Implementation
Week 8	Implement Result Module
Week 9	Perform Security Enhancements & Testing
Week 10	Testing & Documentation

- **Targeted User:**

- **Remote Voters:**

Remote voters are individuals who are eligible to vote but are unable to be physically present at the polling station on election day. This may be due to various reasons such as living far away from their registered voting area, having a physical disability, or facing time constraints caused by work or personal commitments.

To ensure their participation in the democratic process, an online voting platform allows them to conveniently and securely cast their votes from any location. This system promotes inclusivity, accessibility, and equal opportunity for all citizens to exercise their right to vote.

- **Admin User:**

Admin users are responsible for managing and overseeing the entire online voting system. They have access to the administrative dashboard, where they can add, update, or remove voter records, manage candidate details, and monitor election activities in real time.

Admins ensure that the voting process runs smoothly, securely, and transparently by verifying voter identities, handling technical issues, and maintaining data integrity. They also generate and analyze election reports after voting concludes. Overall, admin users play a crucial role in maintaining system efficiency, security, and fairness throughout the online voting process.

• **Requirement Determination**

- Define system requirements (functional and non-functional).
- Collect feedback from potential users.
- Collect data from other voting websites and mentor.
- Set security, performance, and usability expectations.

Requirement Specification

Functional Requirements:

1. Registration and Login Module

Registration Phase

- **Identity Verification:**

Each voter registers using their **Aadhaar Card number** and a **secure password**. The Aadhaar number serves as a unique identifier, ensuring that each citizen can register only once, thus preventing duplicate or fraudulent accounts.

- **Account Creation:**

Once verified, the system automatically generates a **unique user account** linked to the voter's encrypted data in the backend. This account will serve as the authenticated digital identity throughout the election process.

- **Symmetric Cryptography Implementation:**

- The system employs **symmetric key cryptography** (such as AES) to encrypt and decrypt sensitive voter information.
- During registration, the voter's personal data (Aadhaar number, password, etc.) is **encrypted** using a single symmetric key.
- The **encrypted ciphertext** is stored securely in the database, while the **encryption key** is stored in a protected file within the server.

- This layered encryption design prevents unauthorized access to personal data, even if the database is compromised.

Login Phase

- During login, the entered credentials are **decrypted using the same symmetric key** and compared against the stored encrypted data in the database.
- This mechanism ensures:
 - **Confidentiality:** Sensitive data remains unreadable without the correct key.
 - **Integrity:** The verification process is resistant to tampering and interception.
 - **Performance Efficiency:** Symmetric encryption allows for rapid, lightweight authentication suitable for high-load systems.

By implementing symmetric cryptography, the system balances **speed, security, and user privacy**, ensuring that user data remains protected throughout the authentication cycle.

2. User Management Module

The **User Management Module** empowers administrators to oversee the entire voter base efficiently.

Core functionalities include:

- **Approval and Rejection of Registrations:**

Admins can review new voter applications, verify details, and either approve or reject registration requests based on validation results.

- **Voter Record Maintenance:**

Allows administrators to edit, update, or remove voter records as needed (e.g., corrections in Aadhaar linkage or demographic details).

- **Monitoring and Auditing:**

Tracks voter registration activity, including timestamps, login patterns, and account status, ensuring transparency and accountability in the registration process.

This module ensures that only **verified and authorized voters** gain access to the system, preventing impersonation and vote duplication.

3. Vote Management Module

The **Vote Management Module** represents the core of the system — ensuring that every vote is securely recorded, immutable, and verifiable.

Key Functionalities:

- **Blockchain-Based Vote Recording:**
 - Each vote is treated as a **transaction** within the blockchain network.
 - When a vote is cast, it is packaged as a **block** containing:
 - Voter's anonymized ID
 - Candidate selection
 - Timestamp
 - Previous block hash
 - The new block is then linked to the existing chain using **SHA-256 hashing**, ensuring that each vote is **permanently and immutably recorded**.
- **Data Immutability and Transparency:**

Once a vote is recorded in the blockchain, it **cannot be altered, duplicated, or deleted**. This guarantees election integrity and protects against data tampering or system manipulation.
- **Privacy and Anonymity:**

Although the blockchain records every vote, voter identities remain **anonymized** using cryptographic techniques to ensure confidentiality.

- **Auditable and Verifiable Ledger:**

The blockchain provides a transparent ledger that election officials can verify without compromising voter privacy, thereby enhancing **trust and accountability** in the voting process.

4. Result Generation Module

The **Result Generation Module** automates the vote counting and result publication process.

Key Highlights:

- Once the voting period concludes, the system initiates an **automatic tallying algorithm** that aggregates votes directly from the blockchain ledger.
- **Automation:** The process requires **no manual intervention**, thereby eliminating human bias, manipulation, or error.
- **Integrity Validation:** Before finalizing results, the system cross-verifies blockchain data to confirm authenticity and completeness.
- **Efficiency:** Results are computed and displayed instantly once the election period ends.

This ensures a **fast, accurate, and trustworthy** declaration of results.

5. Announcements Module

The **Announcements Module** serves as a **communication bridge** between election administrators and registered voters.

Functions include:

- **Official Notifications:** Broadcast key updates such as:
 - Election start and end dates
 - Result publication dates
 - System maintenance or downtime notices
- **Policy and Regulatory Updates:** Inform voters about changes in election procedures, data privacy policies, or new system features.
- **Visibility and Accessibility:** All announcements are displayed prominently within the user dashboard to ensure that voters remain informed throughout the election process.

6. Admin Control Panel

The **Admin Control Panel** is a centralized management interface designed for **election officials and system administrators**. It provides comprehensive control over all aspects of the digital election system.

Key Functionalities:

- **Election Configuration:**
 - Set up and manage election parameters such as election title, start and end dates, and participating candidates.
 - Modify configurations dynamically while maintaining blockchain consistency.

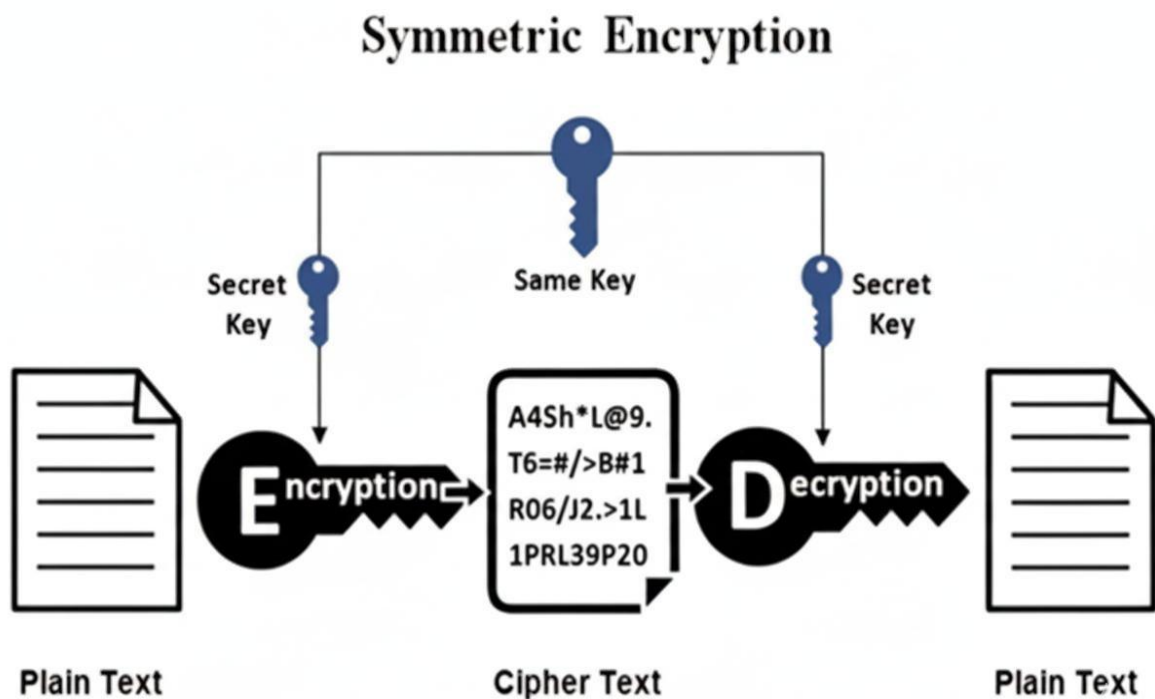
- **Announcement Management:**
 - Create, update, and delete announcements visible to voters.
 - Schedule messages for specific election phases (e.g., voting start reminders).
- **Voter and Candidate Management:**
 - Review and authorize voter registrations.
 - Add or modify candidate profiles, party details, and election participation records.
- **Result Authorization:**
 - Review the automatically generated results.
 - Validate and officially publish them after verification.
 - Generate detailed result reports in downloadable formats (CSV, PDF, etc.).

This module provides **complete operational control** while maintaining **security, transparency, and auditability** across all system components.

Non-Functional Requirements

- **High data security and privacy:**
 - **Symmetric Cryptography Layer:**

The system uses symmetric cryptography to protect voter data. In symmetric encryption, the same key is used to encrypt and decrypt information, making it fast and efficient. This ensures that sensitive data—like voter identity and vote choices—remains confidential and tamper-proof during transmission and storage.



Plain Text -

98765432112

Key -

KrAhTWGwsMPoX7f8ArDuTwtmVCNFRGTdorKsxhOPutk=

Encrypted Text –

gAAAAABopIEql8HFZvHXUTHCBErHi7C6NHP28NUYzYAf5mForG
sxvUdlY8ZbBzdzTHA66ByodCcJAqm1D2bGGIFQBHY6S3PcKLV1K3
MqZXcDNGE_lqcBTNc=

➤ Vote storing security:

```
Block: 1
Hash: 0000bb1088246de2b7bb1e23a038a956b89006944dbbcf266b1b39d023bd296d
Previous :0000000000000000000000000000000000000000000000000000000000000000
Data :Hitesh BJP
Nonce :77010

Block: 2
Hash: 0000d76d81a32e30c242371970b4edad4f0011b6b1f77a6c6e9090402f21b987
Previous :0000bb1088246de2b7bb1e23a038a956b89006944dbbcf266b1b39d023bd296d
Data :Riya BJP
Nonce :97873
```

Block:

Represents the unique position or index of a block within the blockchain.

Data:

This is the actual content stored in the block. In a voting system, this may include anonymized voting information, voter ID reference (not the actual identity), id of election party which user voted.

Hash:

A **SHA-256** cryptographic hash is generated from the block's contents, including the data, previous hash, and **nonce**. This hash acts as a **unique fingerprint** for the block—any change in the block's content, even a single character, will result in a completely different hash.

To add a layer of security and integrity, the system applies a **Proof-of-Work** mechanism with a **0x4 difficulty level**, meaning the generated hash must begin with four leading zeros. Achieving this requires computational effort by adjusting the nonce value repeatedly until a valid hash is found. This ensures that each block is **tamper-resistant**, as altering any block requires re-mining all subsequent blocks in the chain.

Previous Hash:

This field stores the hash of the previous block in the chain, creating a cryptographic link between them. It ensures the integrity and continuity of the blockchain. If any data in a previous block is tampered with, its hash changes, immediately breaking the chain and exposing manipulation.

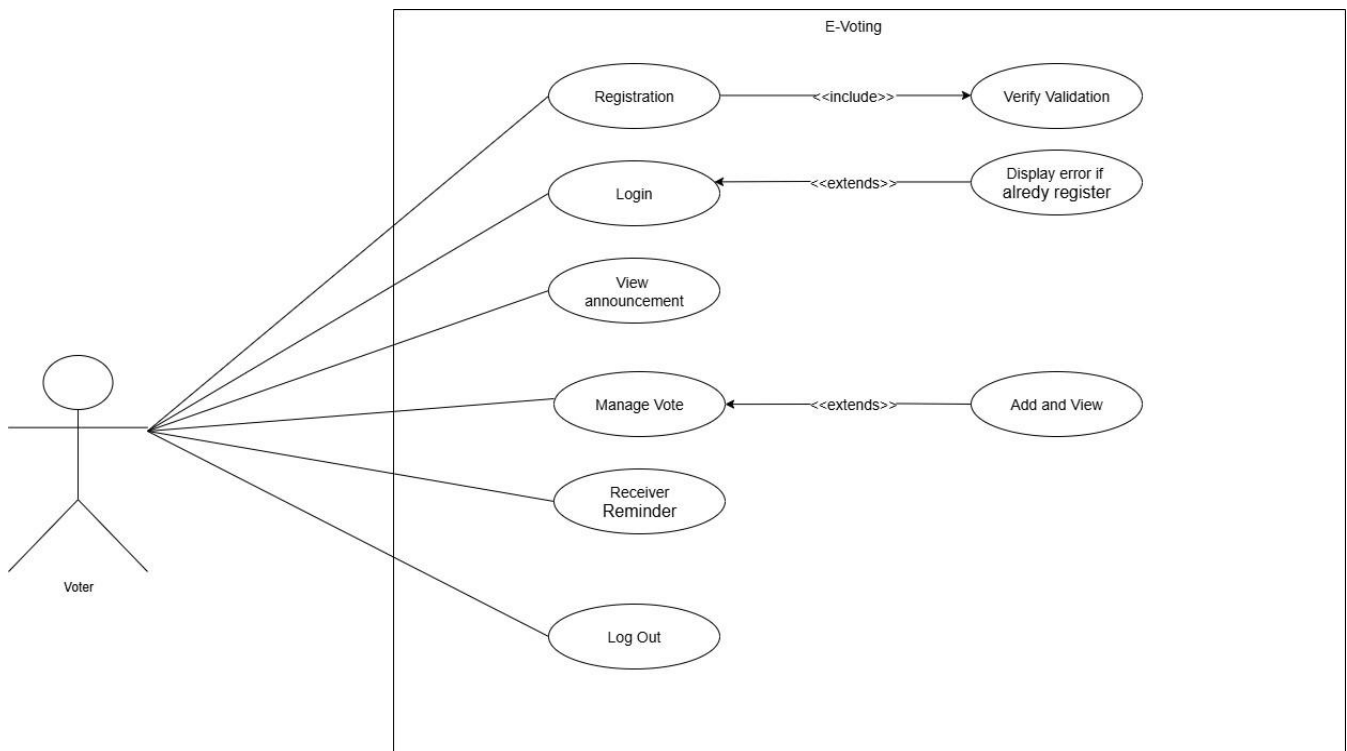
Nonce:

A number that miners (or the system) change repeatedly to find a valid hash that satisfies the difficulty requirement. It's a key component of the mining process, as altering the nonce is what enables the system to generate a valid hash. The nonce ensures that the hash meets the required difficulty level and makes the blockchain resistant to tampering.

Use Case Diagram

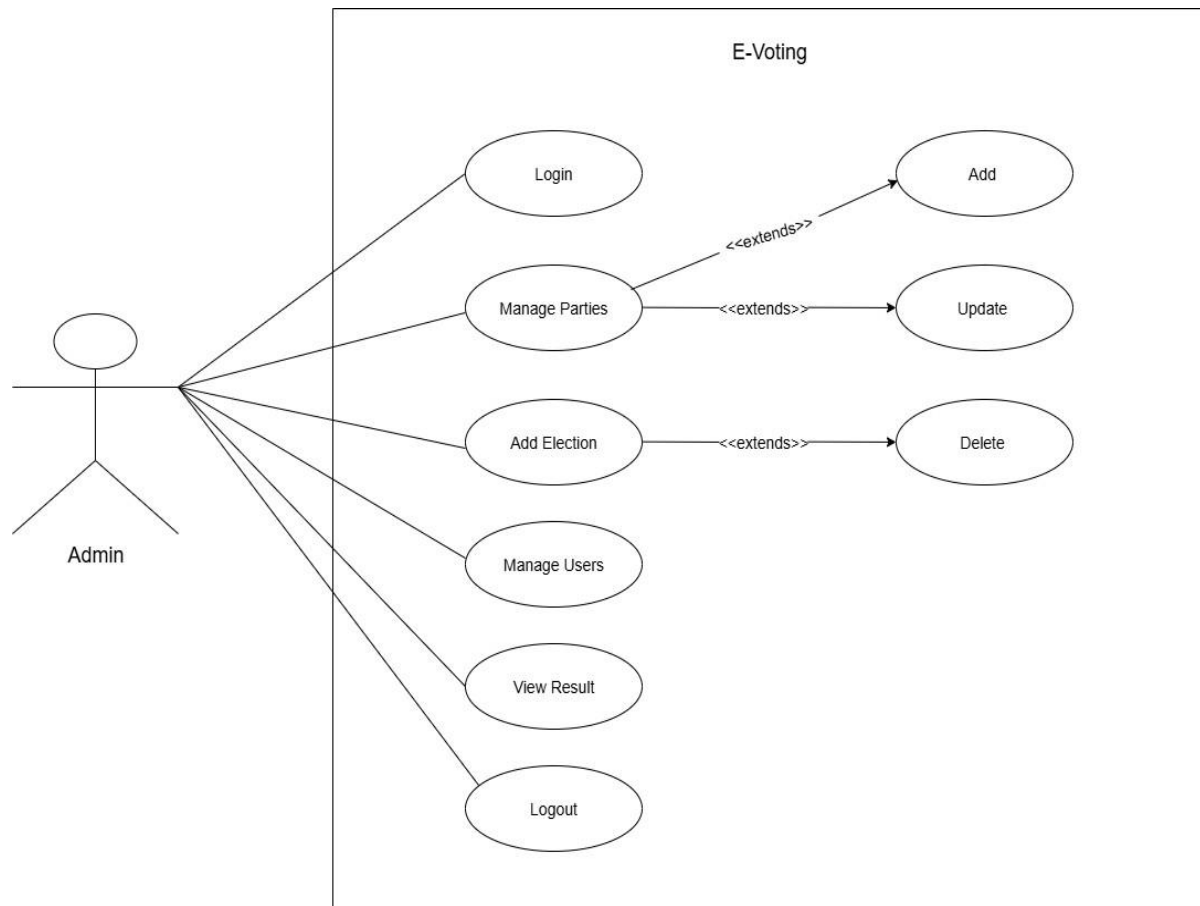
- **Use Cases:**

This use case diagram illustrates the interaction between the **Voter** and the **E-Voting System**



The voter performs key activities such as **Registration, Login, Viewing Announcements, Managing Votes, Receiving Reminders, and Logging Out**. During registration, the system includes a **Verify Validation** process to confirm voter identity and prevent duplicate entries. If a user attempts to register again, an **error message** is displayed. The **Manage Vote** use case extends to **Add and View**, allowing voters to cast and review their votes. Overall, this diagram models the secure, interactive, and user-friendly operations of an online voting system for remote voters.

Admin:



This use case diagram represents the interaction between the **Admin** and the **E-Voting System**. The admin performs essential operations such as **Login**, **Manage Parties**, **Add Election**, **Manage Users**, **View Result**, and **Logout**. The **Manage Parties** use case extends to **Add** and **Update**, allowing the admin to create or modify party details. Similarly.

Data Dictionary

Table: States

Column	Data Type	Constraints	Description
id	Integer (PK)	Auto-generated	Unique identifier for each state
sname	Varchar(40)	Not Null	Name of the state

The States table is a fundamental reference table that maintains a list of all states used within the system. It acts as a **master data table**, ensuring data consistency across various modules—such as voter registration, election configuration, and regional reporting. Each state entry is uniquely identified by a system-generated ID, allowing the application to establish relationships with other tables (e.g., Voters, Elections, Candidates) through foreign keys.

Table: Users

Column	Data Type	Constraints	Description
id	Integer (PK)	Auto-generated	Unique identifier for each user
adhar_card	Varchar (20)	Nullable, Blank allowed	User's Aadhaar card number (optional)
password	Varchar (200)	Not Null	Hashed user password
key	Varchar (50)	Nullable, Blank allowed	Security/verification key (optional)
sid_id	Foreign Key → States.id	Not Null, Cascade delete	Reference to user's state
created_at	Date	Auto-set on update	Timestamp of last update/creation

The **Users** table serves as the **core repository** for storing all registered user information within the blockchain-based election management system.

Each record corresponds to a single system user (voter or admin), containing personal identification details, authentication credentials, and metadata related to account creation and updates.

This table is **directly linked** to the States table through a foreign key (sid_id), ensuring that every user is associated with a specific geographical region or state.

It is a critical component for managing voter registration, authentication, and authorization securely.

Table: Announcement

Column	Data Type	Constraints	Description
id	Integer (PK)	Auto-generated	Unique identifier for announcement
title	Varchar (20)	Not Null	Title of announcement
description	Varchar (100)	Not Null	Detailed description
created_at	Date	Auto-set on update	Timestamp of announcement creation
sid	Foreign Key → States.id	Nullable, Blank allowed	Targeted state for announcement (optional)

The **Announcement** table is used to manage and store all official notifications, updates, and public messages within the blockchain-based election management system.

Announcements are created by administrators to inform users about **important events, election timelines, results, or policy updates**.

This table ensures that every announcement is stored with a timestamp and can be optionally associated with a **specific state**, allowing both **global** and **state-level** communication.

Table: Parties

Column	Data Type	Constraints	Description
id	Integer (PK)	Auto-generated	Unique identifier for each party
name	Varchar (30)	Not Null	Full name of the political party
short_form	Varchar (3)	Not Null	Abbreviation (short form) of party

The **Parties** table stores detailed information about all political parties participating in the election system.

Each record represents a registered political party, including its **official full name** and **standard abbreviation** (short form).

This table acts as a **master reference** for other modules such as **Candidates**, **Elections**, and **Vote Management**, ensuring consistent and standardized use of party information throughout the system.

Table: Election

Column	Data Type	Constraints	Description
Id	Integer (PK)	Auto-generated	Unique identifier for election record
Pid	Foreign Key → Parties.id	Not Null, Cascade delete	Political party reference
Vid	Text	Not Null	Voter IDs (possibly serialized list of voters)
Sid	Foreign Key → States.id	Not Null, Cascade delete	State where election is conducted
vote_count	Integer	Default = 0	Number of votes received
start_date	DateTime	Not Null	Election start date & time
end_date	DateTime	Not Null	Election end date & time

The **Election** table serves as the central repository for managing and tracking all elections within the digital voting system.

It maintains the essential details of each election instance, including the **participating political party**, **associated state**, **voter participation**, and **voting timeline**.

This table integrates with other core modules — such as **Parties**, **States**, and **Blockchain Vote Recording** — to enable transparent, verifiable, and secure election operations.

User Interface Design

Sign up

The screenshot shows a web browser window with the address bar displaying 'http://127.0.0.1:8000'. The page content is a 'Sign up' form for 'vote-Chain'. The form is centered on a light blue background. It contains three input fields: 'Aadhar number' with the value '98765432112', 'Password' with masked characters '****', and 'State' with a dropdown menu showing 'Gujarat'. Below these fields is a prominent blue 'Sign Up' button. At the bottom of the form, there is a link that says 'Already have an Account? Sign In'.

1. User Input Collection

During registration, the voter provides:

- Aadhaar Card Number:** Serves as the unique identity verification parameter.
- Password:** Chosen by the user for authentication.
- State:** Selected from a predefined list (States table) representing their residential state.

2. Identity Verification

The system verifies the provided **Aadhaar Card Number** for format validity (12-digit numeric structure).

Optionally, it can be cross-referenced with a government or mock validation service (depending on implementation scope).

3. Password Encryption

Before storage, the entered password is **encrypted using symmetric cryptography** (e.g., AES algorithm).

The encryption key is **generated dynamically** during registration.

- **Process:**

- a. The password text is converted into ciphertext using the encryption key.
- b. The **encrypted password** is stored in the database (Users.password column).
- c. The **encryption key** is securely stored alongside the data in a protected file (key vault).

This ensures that even if the database is compromised, the raw passwords remain inaccessible and unreadable.

4. Account Creation

After encryption and verification, the system creates a new user record in the **Users** table with:

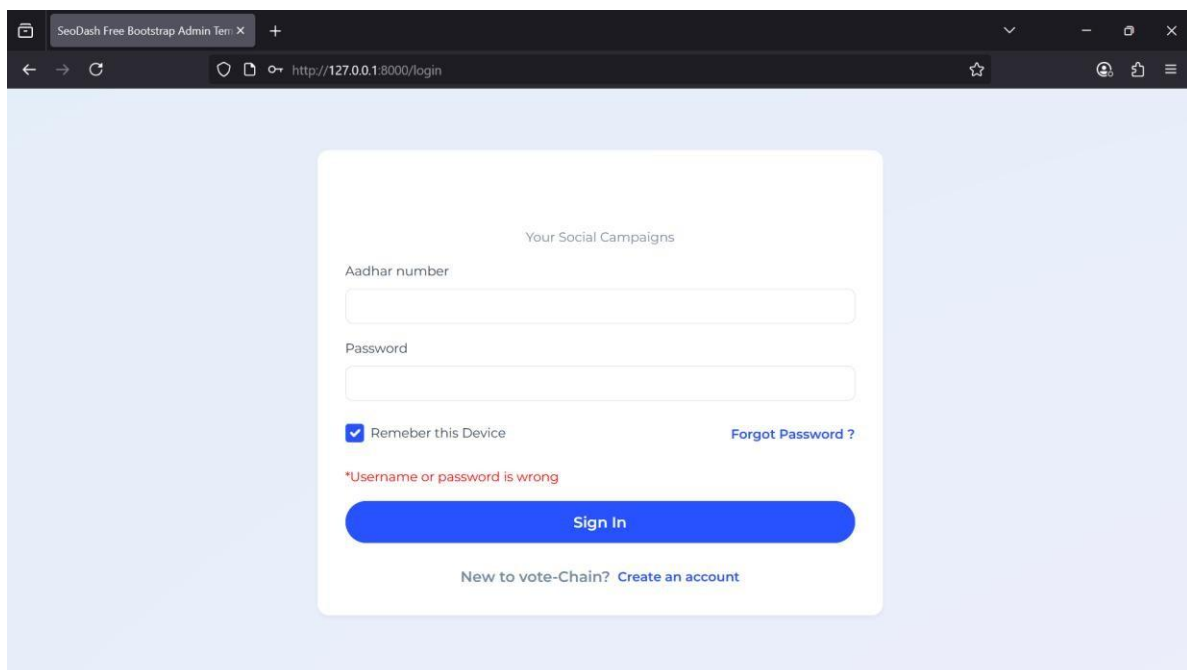
- a. Aadhaar number (adhar_card)
- b. Encrypted password (password)
- c. Encryption key reference (key)
- d. State reference (sid_id)

5. Success Confirmation

The user receives a confirmation message upon successful registration.

A record of the timestamp is maintained (created_at) for audit and tracking.

Sign in



SeoDash Free Bootstrap Admin Tem: X

http://127.0.0.1:8000/login

Your Social Campaigns

Aadhar number

Password

☒ Remeber this Device [Forgot Password ?](#)

*Username or password is wrong

Sign In

New to vote-Chain? [Create an account](#)

1. Credential Submission

- The user provides their **Aadhaar Card Number** and **Password** during login.

2. Decryption & Verification

- a. The system retrieves the corresponding encrypted password and key from the database.
- b. The stored ciphertext is **decrypted** using the same symmetric key originally used during registration.
- c. The decrypted password is compared against the password entered by the user.

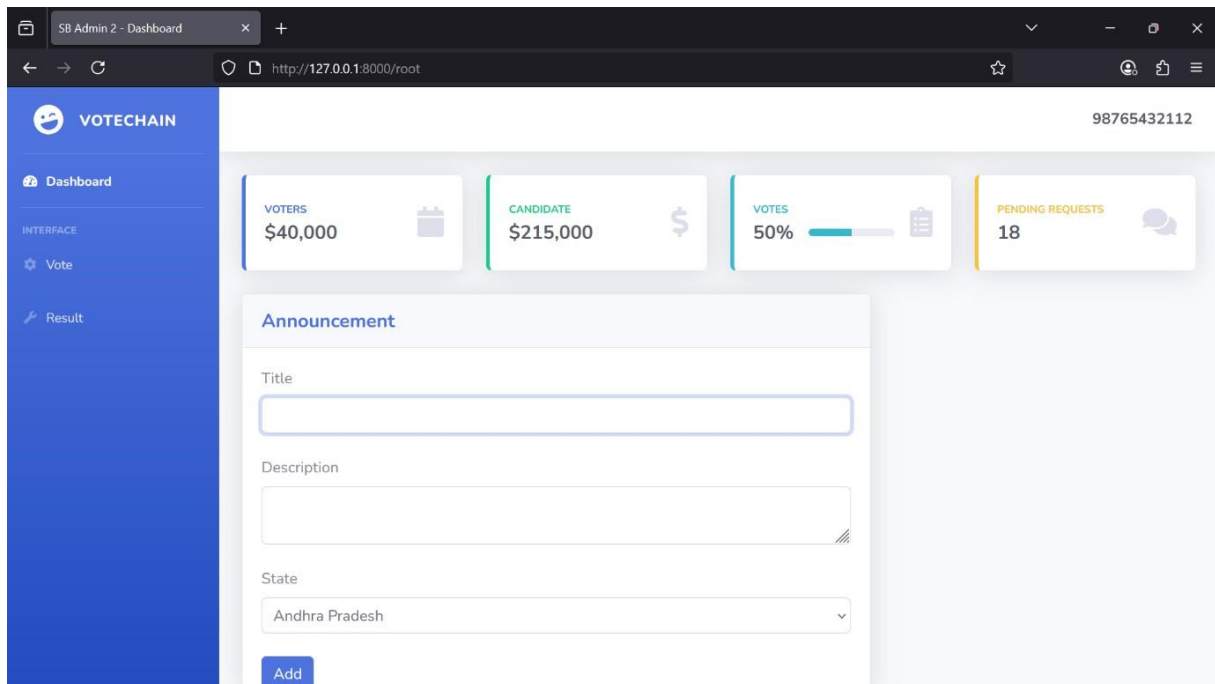
3. Access Grant

- a. If both credentials match:
 - i. The user session is initialized.
 - ii. Access is granted to the system dashboard (voter or admin view based on role).
- b. If credentials do not match:
 - i. The login attempt is rejected with an appropriate error message.

4. Session Management

- a. User sessions are maintained securely using Django's session management system.
- b. Idle sessions automatically expire after a predefined timeout to prevent unauthorized access.

Admin dashboard



This image shows the **Admin Dashboard** of the VoteChain online voting system. The dashboard provides a clear and interactive interface for administrators to manage and monitor election activities. It displays key metrics such as the total number of voters, candidates, voting percentage, and pending requests, helping the admin track real-time progress.

The left-side navigation panel includes options like “Vote” and “Result” for easy access to different modules. The **Announcement** section allows the admin to create new announcements by entering a title, description, and selecting the relevant state. Once filled, the admin can click the **Add** button to publish the update.

Add Election

The screenshot displays the 'Add Election' interface. On the left is a blue sidebar with the title 'INTERFACE' and four menu items: 'Party', 'Candidate', 'Election', and 'View Elections'. The main content area is titled 'Election' and contains a table with two visible rows of political parties. Each row has columns for 'Party Logo', 'Party Name', and 'Select'. Below the table are three input fields: 'State' (a dropdown menu currently showing 'Andhra Pradesh'), 'Start Date & Time' (a date and time picker), and 'End Date & Time' (another date and time picker).

Party Logo	Party Name	Select
	Bharatiya Janta Party (BJP)	☐
	Aam Aadmi Party (AAP)	☐

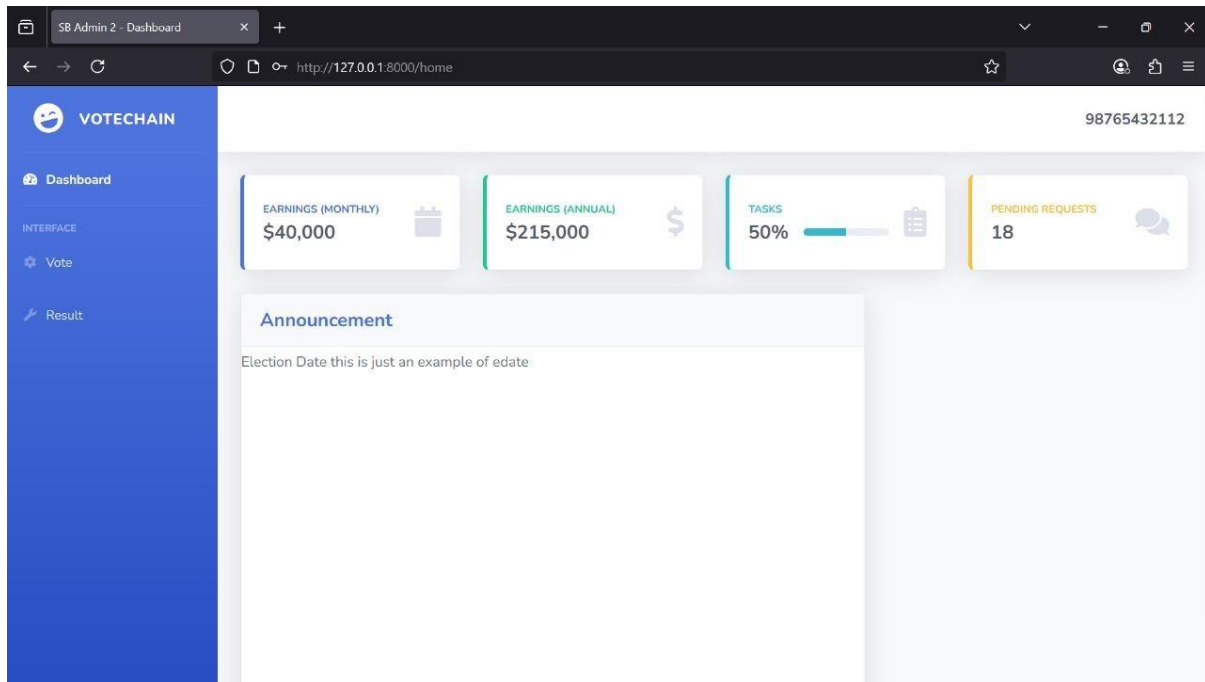
State
Andhra Pradesh

Start Date & Time
dd/mm/yyyy, --:-- --

End Date & Time
dd/mm/yyyy, --:-- --

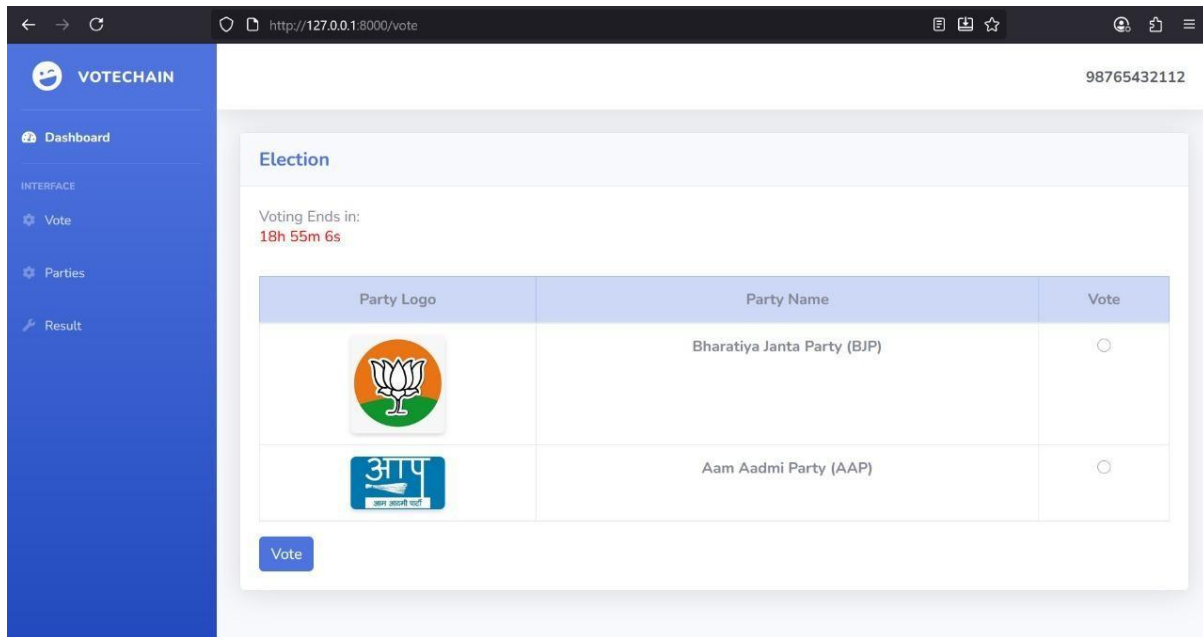
This image represents the **Election Management Page** of the VoteChain system, where the admin can create and manage election events. It displays a list of political parties with their respective logos and names, such as the Bharatiya Janta Party (BJP) and Aam Aadmi Party (AAP).

The admin can select one or more parties to include in the election. Below this, there are input fields to select the **state** (e.g., Andhra Pradesh) and specify the **start** and **end date & time** for the election. This interface allows the admin to efficiently schedule, organize, and configure elections within the online voting system.



This image shows the **UserDashboard** of the VoteChain application, designed to provide a quick overview of essential statistics and updates. The dashboard displays key metrics such as monthly earnings, annual earnings, task completion percentage, and pending requests. These data cards help the admin monitor system activities effectively.

The sidebar menu on the left includes options like **Vote** and **Result**, offering easy navigation between different modules. The **Announcement** section allows the admin to share important updates, such as election dates or other notifications. In this example, it displays a sample message regarding the election date to demonstrate system functionality.



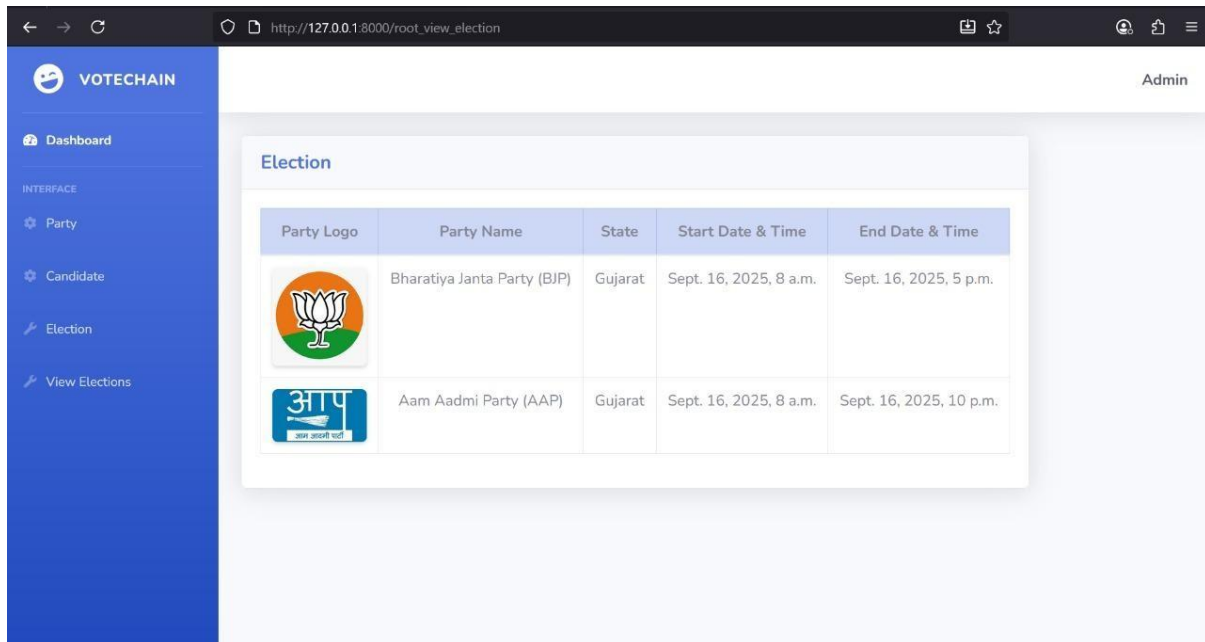
This image displays the **Voting Page** of the VoteChain online voting system, where registered voters can cast their votes securely. The interface shows the list of participating political parties along with their official logos and names, such as the **Bharatiya Janta Party (BJP)** and the **Aam Aadmi Party (AAP)**.

Each party has a corresponding radio button that allows the voter to select their preferred choice. At the top, there is a countdown timer indicating the remaining voting time. Once the voter makes a selection, they can click the **Vote** button to submit their vote digitally in a secure manner.



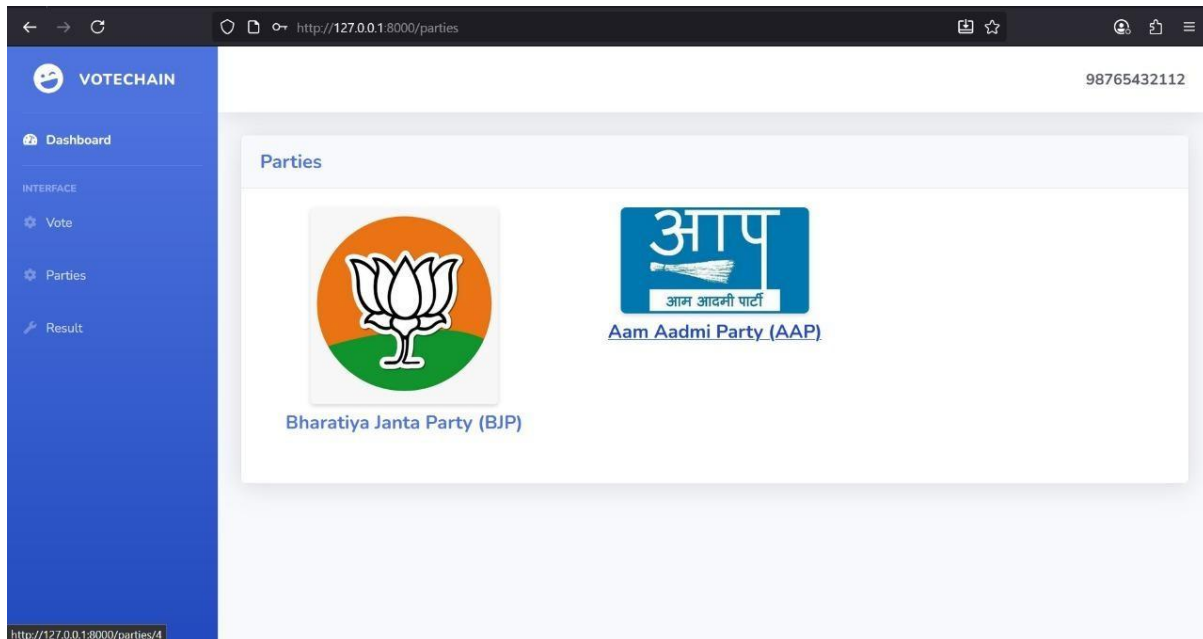
This image shows the **Political Party Management Page** of the VoteChain online voting platform. It is designed for the admin to add and manage political parties that will participate in elections. The form includes fields for entering the **Party Name**, **Short Form**, and uploading the **Party Logo** using the “Browse” button.

After filling in the details, the admin can click the **Add** button to save the party information into the system database. The lower section of the page displays the existing political parties with their respective logos, such as the Bharatiya Janta Party (BJP) and Aam Aadmi Party (AAP).



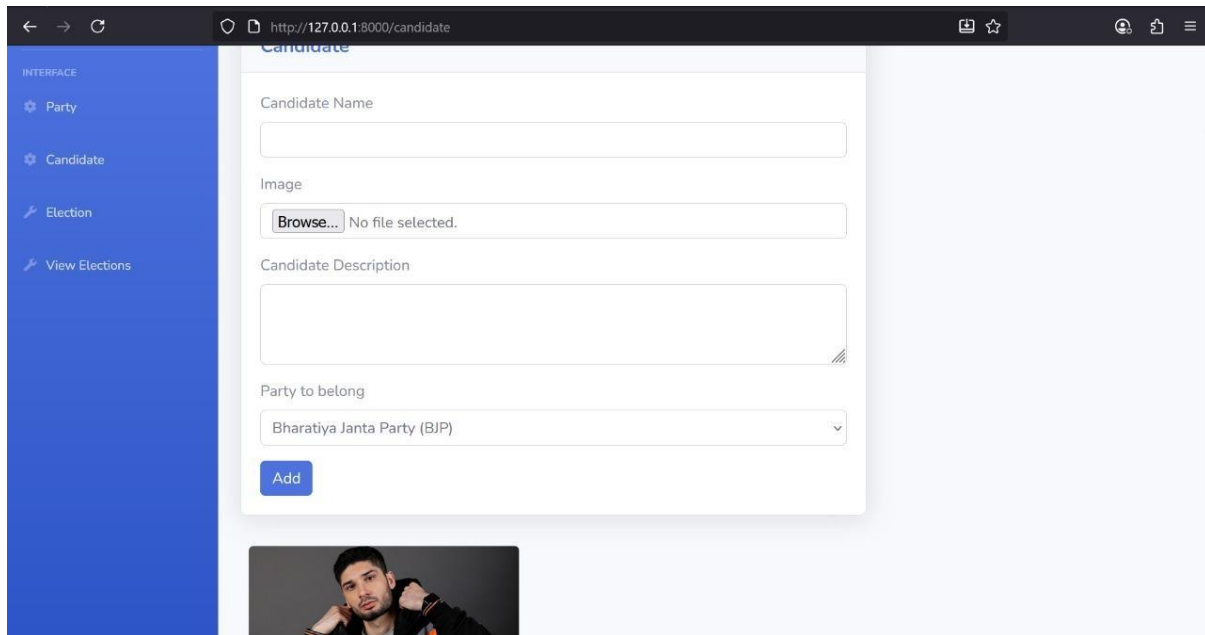
This image displays the **View Elections Page** of the VoteChain system, designed for the admin to review and manage scheduled elections. The page lists all the elections created within the platform, showing key details such as the **Party Logo**, **Party Name**, **State**, **Start Date & Time**, and **End Date & Time**.

In this example, two political parties—**Bharatiya Janta Party (BJP)** and **Aam Aadmi Party (AAP)**—are registered for the state of **Gujarat** with elections scheduled on **September 16, 2025**, between **8 a.m.** and **10 p.m.**. This interface ensures organized tracking and easy management of election events.



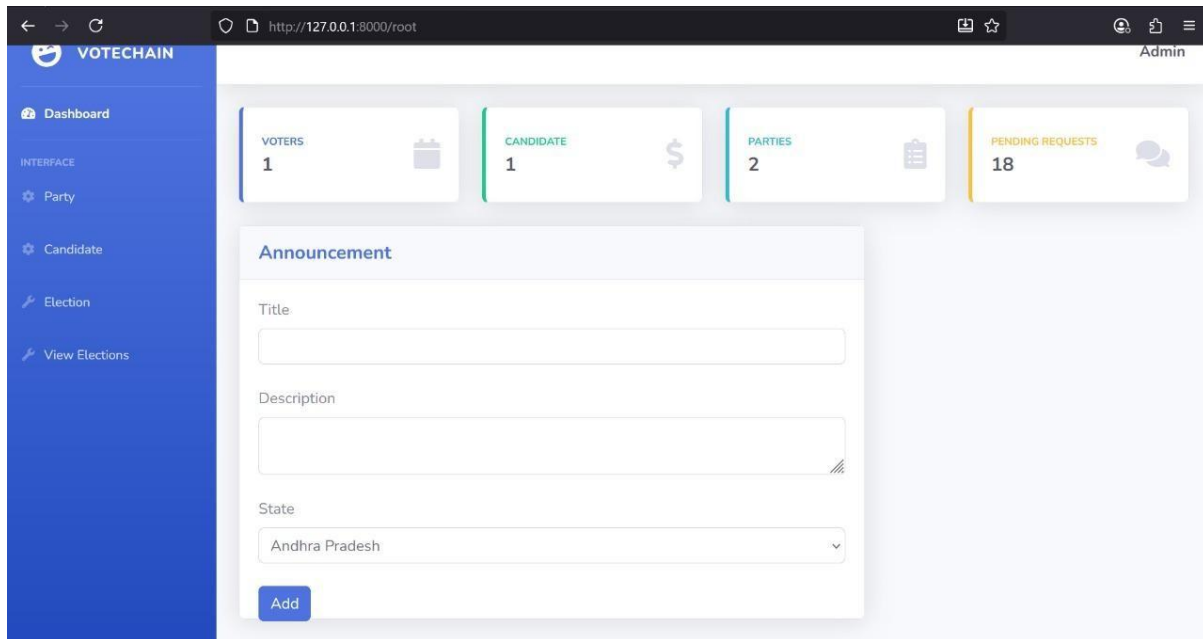
This image represents the **Parties Page** of the VoteChain online voting system. It displays the list of all political parties that are registered and available for voting within the platform. Each party is shown with its **official logo** and **name**, making it easy for voters to identify them visually.

In this example, the page features two political parties: the **Bharatiya Janta Party (BJP)** and the **Aam Aadmi Party (AAP)**. The layout is clean and user-friendly, allowing voters to quickly view and understand their options before casting their vote. The sidebar provides navigation links for Vote and Result modules.



This image shows the **Candidate Management Page** of the VoteChain system, where the admin can add and manage election candidates. The form includes input fields for entering the **Candidate Name**, uploading a **Profile Image**, and writing a **Candidate Description**.

Additionally, there is a dropdown menu to select the **Political Party** to which the candidate belongs, such as the **Bharatiya Janta Party (BJP)**. After filling in the required details, the admin can click the **Add** button to save the candidate's information. The lower section of the page displays the uploaded candidates along with their profile images for easy reference.



This image represents the **Admin Dashboard** of the VoteChain online voting system. It provides an overview of essential statistics, including the total number of voters, candidates, registered parties, and pending requests. The dashboard allows the admin to monitor election-related data in real time.

Below the statistics section, there is an **Announcement** panel where the admin can add updates or notifications. The form includes fields for the **Title**, **Description**, and **State** selection (e.g., Andhra Pradesh). After entering the details, the admin can click the **Add** button to publish the announcement, ensuring all users are informed about important updates.

- Database Records

Django administration

WELCOME, ROOT. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > [E votingclient](#) > [Announcements](#) > [Announcement object \(2\)](#)

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

Users

+ Add

EVOTINGCLIENT

Announcements

+ Add

Candidatess

+ Add

Elections

+ Add

Partiess

+ Add

Statess

+ Add

Userss

+ Add

Change announcement

HISTORY

Announcement object (2)

Title:

Election Date

Description:

this is just an example of edate

Sid:

States object (7)

SAVE

Save and add another

Save and continue editing

Change announcement

Announcement object (2)

Groups

+ Add

Users

+ Add

Announcements

+ Add

Candidatess

+ Add

Elections

+ Add

Partiess

+ Add

Statess

+ Add

Userss

+ Add

Change announcement

Announcement object (2)

Groups

+ Add

Users

+ Add

Announcements

+ Add

Candidatess

+ Add

Elections

+ Add

Partiess

+ Add

Statess

+ Add

Userss

+ Add

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

LMCA

Vote Chain

54 | Page

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + AddUsers + Add

EVOTINGCLIENT

Announcements + AddCandidatess + AddElections + AddPartiess + AddStatess + AddUserss + Add

Change candidates

HISTORY

Candidates object (1)

Cname: Dinesh Thakor

Cphoto: Currently: static/images/KRna_EGF9XAz.png
Change: Choose File No file chosen

Description: He is ddcjhbawjwhdcbsdsw awa wawwaeuwdee wejdb

Pid: Parties object (3)   

SAVE

Save and add another

Save and continue editing

Delete

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

EVOTINGCLIENT

Announcements + Add

Candidatess + Add

Elections + Add

Partiess + Add

Statess + Add

Userss + Add

Change election

HISTORY

Election object (7)

Pid:Parties object (3) +

Sid:States object (7) +

Vid:8,

Vote count:1

Start date:

Date:2025-11-12Today

Time:12:20:05Now

Note: You are 5.5 hours ahead of server time.

LMCA

Vote Chain

56 | Page

[Home](#) > [Evotingclient](#) > Elections

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

[Groups](#) [+ Add](#)[Users](#) [+ Add](#)

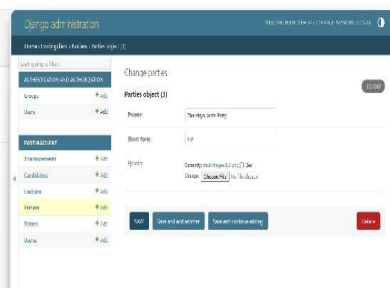
EVOTINGCLIENT

[Announcements](#) [+ Add](#)[Candidates](#) [+ Add](#)[Elections](#) [+ Add](#)[Partiess](#) [+ Add](#)[Statess](#) [+ Add](#)[Userss](#) [+ Add](#)

Select election to change

ADD ELECTION [+](#)Action: 0 of 2 selected☐ ELECTION☐ Election object (8)☐ Election object (7)

2 elections



Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

127.0.0.1:8000/admin/evotingclient/candidates/add/



Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

[Groups](#) [+ Add](#)[Users](#) [+ Add](#)

EVOTINGCLIENT

[Announcements](#) [+ Add](#)[Candidatess](#) [+ Add](#)[Elections](#) [+ Add](#)[Partiess](#) [+ Add](#)[Statess](#) [+ Add](#)[Userss](#) [+ Add](#)

Change parties

HISTORY

Parties object (3)

Pname: Short form: Pphoto: Currently: static/images/BJP.png [Clear](#)
Change: No file chosen

SAVE

Save and add another

Save and continue editing

Delete

127.0.0.1:8000/admin/auth/user/add/



Start typing to filter...

AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add
Users	+ Add
EVOTINGCLIENT	
Announcements	+ Add
Candidatess	+ Add
Elections	+ Add
Partiess	+ Add
Statess	+ Add
Userss	+ Add

Change users

HISTORY

Users object (6)

Adhar card: 98765432112

Password: gAAAAABowDapM6S6Klc2xXM8C2ZaEBOBH

Key: 5WpuAMV_RKu2i9Hxjy24LNIZxOhUaxlBZHHu

Sid id: States object (7)   

SAVE

Save and add another

Save and continue editing

Delete

AUTHENTICATION AND AUTHORIZATION

[Groups](#) [+ Add](#)[Users](#) [+ Add](#)

EVOTINGCLIENT

[Announcements](#) [+ Add](#)[Candidatess](#) [+ Add](#)[Elections](#) [+ Add](#)[Partiess](#) [+ Add](#)[Statess](#) [+ Add](#)[Userss](#) [+ Add](#)

Change states

States object (28)

[HISTORY](#)

Sname:

[SAVE](#)[Save and add another](#)[Save and continue editing](#)[Delete](#)

Report Design

Total numbers of voters – 1,30,20,200

Total number of parties - 6

Total number of candidates - 2,000

Total Count by Parties for specific year - (2023)

Party	Total Votes
Party A (Liberal)	3,450,000
Party B (Conserv.)	2,980,000
Party C (Green)	865,000
Party D (Socialist)	540,000
Independent	210,000
Others	95,000

Coding Standards

(Python/Django)

Models:

Registration view:

```
def register(request):
    if request.method=="POST":
        aadharno=request.POST.get('aadharno')
        txtpassword=request.POST.get('password')
        state=request.POST.get('state')
        st_state=States.objects.get(id=state)

        password=Crypto(txtpassword)
        key=password.generate_key()
        encrypted_pass=password.encryptdata(key)
        user=Users(adhar_card=aadharno,password=encrypted_pass,key=key,sid_id=st_state)
        user.save()

        return HttpResponseRedirect(reverse('login'))

states=States.objects.all()
return render(request,'auth-register.html',context={"states":states})
```

This Python Django function handles user registration. It retrieves Aadhaar number, password, and state from an HTTP POST request, encrypts the password using a custom Crypto class, and saves user details in the database. After registration, it redirects the user to the login page. All available states are passed to the registration template.

Login View:

This Python Django function manages user login authentication. When a POST request is made, it retrieves the Aadhaar number and password from the form. It then checks if the Aadhaar number exists in the database. If found, the function decrypts the stored encrypted password using the user's unique key and compares it with the entered password. If both match, the user's Aadhaar number is stored in the session, and the user is redirected to the home page. If authentication fails, an appropriate error message is displayed, and the user is redirected to the login page. Otherwise, the login form is rendered.

User_vote:

This Django function handles the user voting process. It retrieves the logged-in user's Aadhaar number from the session and verifies eligibility. When a POST request is made, it captures the selected election ID, increments the vote count, and appends the user ID to the election's voter list. It also records the vote details (user ID, election ID, and timestamp) in a blockchain to ensure transparency and immutability. The blockchain mines a new block for each vote and displays the updated chain. The function checks if the user has already voted and renders the election page with voting status and remaining duration.

User Parties:

This Django function, **user_parties**, is responsible for displaying a list of all political parties to the user. It retrieves all records from the **Parties** model using the ORM query `Parties.objects.all()` and stores them in the variable `parties`.

These records are then passed into a context dictionary, which is sent to the **user_parties.html** template for rendering. The template uses this data to present the party names and related details dynamically. This function plays a crucial role in allowing users to view available parties before participating in an election or casting their votes, ensuring better transparency and accessibility in the voting system.

User Candidates:

This Django function, **user_candidates**, displays a list of candidates belonging to a specific political party. It takes the HTTP request and a party ID (`id`) as parameters. Using the ORM query `Candidates.objects.filter(pid=id)`, it retrieves all candidates associated with the given party ID from the database. The results are stored in the `candidates` variable and passed into a context dictionary.

Finally, this context is rendered to the **user_candidates.html** template, where the candidate details are dynamically displayed to users. This function helps voters view the candidates representing each party, enabling informed decision-making during the voting process.

Admin add election:

This Django view function `admin_add_election(request)` allows an admin to add new elections. When a POST request is made, it collects form data — selected party IDs (`pid`), state ID (`state`), start date (`sdate`), and end date (`edate`). It retrieves the corresponding State object, then loops through all selected parties to create new election entries in the Election table using the provided details. After successful insertion, it redirects to `"root_add_election"`. For GET requests, it fetches all Parties and States, prepares them in the context dictionary, and renders the `admin_add_election.html` page for the admin interface.

Admin Dashboard:

This Django view function `admin_dahsboard(request)` manages the admin dashboard. When a POST request is received, it collects input values such as title, description, and state from the form. It retrieves the corresponding State object and creates a new Announcement record with the given details, saving it to the database. After saving, it redirects to `"root_dahsboard"`. For normal page loads (GET requests), it gathers data like all states, total users, total candidates, and total parties using model queries. These statistics are passed through the context dictionary and rendered on the `admin_dashboard.html` page for administrative overview and control.

Admin add Candidate:

This Django view function `admin_add_candidate(request)` enables the admin to add new candidates. When the method is POST, it retrieves form data including the candidate's name (`cname`), photo (`cphoto`), description (`desc`), and selected party (`party`). It fetches the corresponding Party object and creates a new Candidate record with these details, saving it to the database. After saving, it fetches all existing Parties and Candidates records to display updated data. The context dictionary containing this information is passed to the `admin_candidate.html` template, which renders the candidate management page for the admin to view, add, and manage candidate information efficiently.

Admin view election:

This Django view function `admin_view_election(request)` is designed to allow administrators to view all election records stored in the database. It retrieves all entries from the Election model using `Election.objects.all()` and stores them in the `elections` variable. This data is then passed into a context dictionary, which is used to dynamically display the election details on the webpage. Finally, the function renders the `admin_view_election.html` template along with the context, enabling the admin to view information such as election IDs, states, participating parties, and dates directly from the web interface. It ensures easy monitoring of election data.

Admin Party:

This Django view function `admin_party(request)` enables the admin to add and manage political parties. When a POST request is made, it retrieves the party name (`pname`), short form (`short_form`), and party photo (`pphoto`) from the submitted form. A new `Parties` object is created with these details and saved to the database. After successfully adding the party, it redirects to `"root_party"`. For GET requests, it fetches all existing parties from the database using `Parties.objects.all()`. These records are then passed into the context dictionary and rendered on the `admin_party.html` template, allowing the admin to view and manage party details easily.

Admin View Result:

This Django view function `admin_view_result(request)` is responsible for displaying election results that are currently active based on date and time. It first captures the current date and time using `datetime.now()`. Then, it filters the `Election` model to retrieve elections whose `start_date` is less than or equal to the current time and whose `end_date` is greater than or equal to it. These elections are stored in the `result` variable. The retrieved data is added to the context dictionary and rendered using the `admin_view_result.html` template, allowing the admin to view and manage ongoing or completed election results effectively.

User Result:

This Django view function `user_result(request)` allows users to view elections that are currently active based on the system date and time. It uses `datetime.now()` to obtain the current time and filters the Election model to retrieve elections where the `start_date` is less than or equal to the current time and the `end_date` is greater than or equal to it. These filtered results are stored in the `result` variable. The data is then passed into the context dictionary and rendered through the `user_result.html` template. This enables users to view real-time ongoing election results conveniently and securely.

URL Configuration for E-Voting System:

```
urlpatterns = [
    path('candidate', views.admin_add_candidate, name="root_candidate"),
    path('root', views.admin_dahsboard, name="root_dahsboard"),
    path('party', views.admin_party, name="root_party"),
    path('root_add_election', views.admin_add_election, name="root_add_election"),
    path('root_view_election', views.admin_view_election, name="root_view_election"),
    path('root_view_result', views.admin_view_result, name="root_view_result"),

    path('parties', views.user_parties, name="user_party"),
    path('parties/<id>', views.user_candidates, name="user_candidates"),
    path('', views.register, name="register"),
    path('login', views.login, name="login"),
    path('home', views.home, name="home"),
    path('vote', views.user_vote, name="user_vote"),
    path('result', views.user_result, name="user_result"),
]
```

This Django urlpatterns list defines all the URL routes for the E-Voting web application, connecting each route to its corresponding view function. Admin-related paths include adding candidates (root_candidate), managing the dashboard (root_dahsboard), handling parties (root_party), and managing elections and results. User-related routes include viewing parties, candidates, voting, and checking results. Additionally, registration and login routes allow users to create accounts and access the system. Each path is mapped to a specific view in the views module and assigned a unique name for easy URL referencing within templates, ensuring smooth navigation and functionality.

Test cases

Below are structured test cases, grouped by type (unit, integration, functional, security, performance, UI/usability). Each case states purpose, preconditions, steps, expected result, and pass criteria.

Unit tests (backend / blockchain functions)

1. Block.hash calculation

- Purpose: Verify `calculate_hash()` produces correct SHA-256 for a given block.
- Precondition: Create `Block(index=1, timestamp="2025-09-16T08:00:00", voter_id="v123", candidate_id="c45", previous_hash="0000abcd")`.
- Steps: Call `calculate_hash()` and compare to expected SHA-256 of concatenated string.
- Expected: Hash equals deterministic SHA-256 value.
- Pass: Function returns identical hex digest.

2. Blockchain.add_block (append & link)

- Purpose: Ensure new block's `previous_hash` equals previous block's hash.
- Precondition: Blockchain with genesis block.
- Steps: Add a vote block, inspect `previous_hash`.
- Expected: `new_block.previous_hash == blockchain[-2].hash`.
- Pass: True.

3. User model password encryption/decryption

- Purpose: Verify symmetric encryption stores and returns correct password after decrypt.
- Precondition: Known symmetric key and plaintext password.
- Steps: Encrypt, store, then decrypt.
- Expected: Decrypted password equals original.
- Pass: True.

4. Election vote_count increment

- Purpose: Confirm vote counter updates when a vote is accepted.
- Precondition: Election with vote_count=0.
- Steps: Cast valid vote, commit transaction.
- Expected: vote_count == 1.
- Pass: True.

Integration tests (Django views + blockchain layer)

1. Full vote submission flow

- Purpose: Voter logs in → selects candidate → vote recorded in DB & blockchain.
- Precondition: Registered voter, active election, candidate exists.
- Steps: Simulate POST /vote with session cookie.

- Expected: DB vote record present; blockchain has new block with anonymized voter ref; election vote_count incremented.

- Pass: All three conditions true.

2. Single-vote enforcement

- Purpose: Ensure a voter cannot vote twice for same election.
- Precondition: Voter has already voted.
- Steps: Attempt second POST /vote.
- Expected: System returns error/forbids vote; no new block added.
- Pass: No change to blockchain/election counts.

3. Admin result view vs blockchain

- Purpose: Admin-generated result matches tally from blockchain.
- Precondition: Completed election with blockchain entries.
- Steps: Request admin results; independently tally blockchain blocks for election.
- Expected: Both tallies match.
- Pass: Counts identical.

Functional / Acceptance tests (end-to-end)

Use these as acceptance criteria for stakeholders.

1. Registration & Aadhaar validation

- Goal: Voter can register using 12-digit Aadhaar; invalid formats rejected.
- Test: Try valid and invalid Aadhaar numbers.
- Expected: Valid accepted; invalid rejected with message.

2. Election lifecycle (create → start → vote → end → results)

- Goal: Admin can create and control election; voters vote only during active window.
- Test: Create election with start/end; try voting before start, during, after.
- Expected: Votes allowed only within window.

3. Announcement visibility

- Goal: Announcements created by admin appear to targeted state users.
- Test: Create global & state-specific announcements; check user dashboard.
- Expected: Correct visibility.

Security tests

1. Tamper detection

- Purpose: Simulate altering an old block; verify chain validation fails.
- Steps: Modify a block's data and run `is_chain_valid()`.
- Expected: Validation returns false; admin alerted.
- Pass: Tamper detected.

2. Password & key storage

- Purpose: Ensure encryption keys are not stored in DB plaintext.
- Test: Inspect DB for raw keys/passwords.
- Expected: No plaintext; keys in protected vault/file with restricted perms.

3. Session expiry

- Purpose: Idle session expires.
- Test: Login, idle beyond timeout, attempt action.
- Expected: Redirect to login.

4. Rate limiting / brute-force protection

- Purpose: Prevent repeated login attempts.
- Test: Repeated failed logins from same IP.
- Expected: Temporary block / CAPTCHA.

5. SQL injection / input sanitization

- Purpose: Ensure inputs are ORM-parameterized.
- Test: Submit SQL meta-characters in forms.

Performance tests

1. Concurrent voting simulation

- Purpose: Measure system behavior under N simultaneous vote submissions.
- Test: Use a load tool (e.g., Locust) with N = 1000 over short window.
- Metrics: Average response time, error rate, DB lock contention, blockchain mining latency.
- Pass thresholds: <2s average response, <1% error.

2. Blockchain mining latency

- Purpose: If proof-of-work used (difficulty 0x4), measure block mining time.
- Test: Time to mine 100 blocks.
- Expected: Acceptable average (tunable); if too slow, change difficulty or use PoS/none.

UI / Usability tests

1. Responsive layout

- Test devices: Desktop, tablet, mobile.
- Expected: Voting page, candidate grid, and admin dashboard render correctly.

2. Accessibility

- Test: Keyboard navigation, screen reader labels, color contrast.
- Expected: WCAG AA compliance for core pages (registration, voting).

Sample Test Case Table (concise)

ID	Area	Test	Expected
UT-01	Unit	Block hash deterministic	Same SHA-256 string
IT-01	Integration	Cast vote end-to-end	DB+blockchain+count updated
FT-01	Functional	Duplicate vote attempt	Rejected; no block added
SEC-01	Security	Alter block data	Chain invalidated
PERF-01	Perf	1000 concurrent votes	<2s avg; <1% errors
UI-01	UI	Mobile vote flow	Usable, no overflow

Proposed Enhancements

1) Replace PoW with lighter consensus or remove mining (Priority: High)

- Why: Proof-of-work with difficulty requiring mining (4 leading zeros) is slow and unnecessary for a permissioned voting system. It creates latency for each vote.
- Proposal: Use a permissioned ledger approach:
 - Option A: Remove PoW and append blocks with digital signatures (trusted admin nodes).
 - Option B: Use a lightweight consensus (Practical Byzantine Fault Tolerance — PBFT) if multiple validators exist.
- Implementation: Modify blockchain module to accept signed blocks; store validator signatures; implement `verify_block_signature()`.

2) Voter identity proofing & multi-factor authentication (MFA) (Priority: High)

- Why: Aadhaar alone can be spoofed. MFA boosts security and compliance.
- Proposal: Add OTP to registered mobile/email and optional biometric (future).
- Implementation: Integrate OTP service (Twilio/SMTP), store hashed OTP tokens for short TTL.

3) Use public-key cryptography for anonymization & vote encryption (Priority: High)

- Why: Symmetric encryption key storage is a risk; public-key allows verifiable encryption without sharing private keys.
- Proposal: Each election generates a public key; votes encrypted with election public key; only authorized tallying authority holds private key or use threshold decryption.
- Implementation: Use RSA/ECC libraries; consider threshold decryption (Shamir/threshold crypto) to prevent single point of decryption.

4) Move from SQLite → PostgreSQL (Priority: Medium)

- Why: Production scalability, concurrency, and integrity features.
- Implementation: Update Django settings, migration, use pgbouncer for connection pooling.

5) Immutable, exportable audit trail & public explorer (Priority: Medium)

- Why: Transparency — allow public to audit anonymized chain.
- Proposal: Read-only web explorer that shows blocks (without voter PII).
- Implementation: Create /explorer view; paginate chain; provide CSV/JSON exports.

6) Role-based distributed validators (Priority: Medium)

- Why: Avoid centralized admin who can alter system.
- Proposal: Allow multiple election authorities/nodes to co-sign/validate blocks.
- Implementation: Add node registry, signature aggregation.

7) Stronger privacy: mixnets / homomorphic tally (Priority: Low / Research)

- Why: Prevent linking of a specific vote to a voter while keeping verifiability.
- Proposal: Research homomorphic encryption or mixnets for vote tallying.
- Implementation: Prototype with existing libraries; note complexity.

8) Mobile-first UI + PWA (Priority: Medium)

- Why: Increase accessibility and offline support.
- Implementation: Rework front-end with responsive-first design and service workers to allow cached ballots (submit when online).

9) Audit & monitoring dashboard (Priority: Medium)

- Why: Real-time alerts for anomalies, tamper attempts.
- Implementation: Integrate Prometheus/Grafana; log tamper checks and failed validations.

Conclusion

Vote Chain demonstrates a practical integration of a Django web app with a custom blockchain to provide a more transparent, tamper-resistant voting workflow. The project successfully models the following:

- Immutable vote recording: Each vote becomes a blockchain block, increasing auditability.
- Authentication & role controls: Django-based user/admin modules manage registration, login, and election lifecycle.
- Usable admin & voter interfaces: Bootstrap-based templates allow admin control and voter access.

Key accomplishments

- A working prototype that connects Django models and views to a blockchain layer.
- End-to-end flow: registration → authentication → voting → blockchain recording → results.

Main limitations

- Current PoW mining can add unacceptable latency for live elections.
- Reliance on symmetric key storage for some sensitive operations is a security risk.
- SQLite limits concurrency in high-load scenarios.

Final recommendations

- For any real-world deployment, adopt a permissioned ledger / signature-based block addition, strengthen identity proofing (MFA/OTP), and migrate to a production DB (Postgres). Implement independent auditors and a public, anonymized explorer for transparency.

Bibliography / References

The bibliography lists documents, standards, and resources useful for development, security, and academic citation.

1. Project report — *Vote Chain* . (Your uploaded project report).
2. Django Documentation — Models, Views, Authentication. Django Project. <https://docs.djangoproject.com/>
3. SHA-256 specification — Secure Hash Standard (FIPS PUB 180-4). NIST.
4. Symmetric Encryption (AES) references — NIST SP 800-38A and common AES implementations (PyCryptodome docs).
5. Blockchain basics — S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System.” (for blockchain principles and immutability concepts).
6. PBFT & Permissioned ledgers — Castro & Liskov, “Practical Byzantine Fault Tolerance” (for consensus alternatives).
7. Homomorphic encryption & e-voting research — academic articles on end-to-end verifiable voting (Benaloh, Chaum, etc.).
8. OWASP Top 10 — Web application security guidelines (for preventing injection, XSS, CSRF, etc.).

References :

[GitHub - codingBeast25/Blockchain-based-File-Storage: Upload and download files using flask. Files and transactions are stored using blockchain.](#)

Future Enhancements

1. Lawful Integration with National ID Systems (e.g., Aadhaar)

Purpose: enable secure, reliable identity verification by integrating VoteChain with an authorized government identity provider (for India, the Aadhaar ecosystem) to reduce duplicate registrations and improve voter authenticity.

Benefits

- Stronger identity assurance, reducing fraud.
- Faster KYC onboarding.
- Improved trust for stakeholders (voters, election officers).

Implementation notes

- Use official, government-provided APIs and SDKs (e.g., UIDAI or authorized KYC/Aadhaar Application Providers). Do **not** attempt any unofficial access.
- Prefer one-way verification calls (e.g., “is this person X?” or OTP-based verification) rather than storing raw ID numbers.
- Store only a minimal, consented identifier (hashed reference) and the verification timestamp — avoid storing the Aadhaar number itself.
- Provide explicit, auditable user consent screens before any identity check.

Security & privacy

- Encrypt any PII at rest with strong AES-256 and in transit with TLS 1.3.
- Use HSMs (Hardware Security Modules) for key management.
- Maintain audit logs of verification requests and accesses. Logs must be tamper-evident (append-only).
- Provide data minimization, retention policies and deletion-on-request.

Regulatory & legal

- Comply with Aadhaar Act and UIDAI regulations (including permitted use-cases and storage rules).
- Implement Consent Management and follow local privacy laws (India’s IT Rules / Draft DP Bill, or GDPR if operating internationally).
- Carry out Data Protection Impact Assessments (DPIA) before rollout.

Dependencies

- Formal authorization from the government or a licensed KYC provider.
- Secure API credentials and SLA with provider.

Acceptance criteria

- Verified user count matches KYC provider responses.
- No raw Aadhaar numbers retained in DB.
- Consent logged for each verification.

Suggested test cases

- Successful OTP verification flow (positive).
- Failed verification (mismatched details).
- Attempt to reuse a verification token (replay should fail)

2. Biometric Authentication (Fingerprint and Face Recognition)

Purpose: add biometric checks to provide strong authentication for voter login and voting signing.

Benefits

- Convenient, quick authentication for users.
- Stronger non-repudiation for critical actions (casting vote).

Implementation notes

- Prefer **on-device biometric verification** (e.g., Android/iOS fingerprint/FaceID) that returns a yes/no from OS — never export raw biometric images.
- If server-side biometrics are needed (e.g., central fingerprint matching), use specialist certified vendors and store only biometric templates, not raw scans.
- Consider combining biometrics with device-binding or a PIN (multi-factor).

Security & privacy

- Avoid storing raw biometric images. Store industry-standard protected templates only.
- Use secure enclaves / keystores to protect templates on-device.
- Make biometric optional; offer alternative authentication for accessibility and legal reasons.

Regulatory & legal

- Biometric data is highly sensitive. Obtain explicit consent and follow applicable biometric data laws (explicit consent, strict retention, allowable purposes).
- Provide opt-out and standard authentication fallback.

Dependencies

- Mobile SDKs for FaceID/Fingerprint or vendor SDK for server-match.
- Compliance review and biometric vendor audit.

Acceptance criteria

- Biometric enrollment and verification succeed for supported devices.
- Biometric templates cannot be exported in plaintext.
- Fall-back authentication works reliably.

Suggested test cases

- Enrollment → verification cycle on multiple devices.
- Enrollment with poor-quality capture (reject & prompt).
- Accessibility fallback (PIN/OAuth).

3. FIDO2 / WebAuthn for Passwordless, Secure Authentication

Purpose: enable passwordless, phishing-resistant authentication using device-bound keys (security keys, platform authenticators).

Benefits

- High security (public-key cryptography).
- Improved UX (fast, passwordless logins).
- Works with biometrics but without sending biometric data to server.

Implementation notes

- Implement WebAuthn (FIDO2) for both web and native apps.
- Support platform authenticators (Touch ID, Windows Hello) and external security keys (YubiKey).
- Provide registration, authentication and attestation flows.

Security & privacy

- Private keys never leave the user's device.
- Store only public keys and attestation metadata.

Dependencies

- Client support: modern browsers and OS-level support.
- Server libraries for WebAuthn (various open-source packages available).

Acceptance criteria

- Users can register a security key and authenticate without passwords.
- Attestation failures are handled gracefully.

Suggested test cases

- Registration with platform and external authenticators.
- Authentication on a new device (should fail without registration).
- Lost-key recovery procedures.

4. Internationalization & Global Launch (Multilingual + Multi-Jurisdiction)

Purpose: adapt VoteChain for use in other countries — supports multiple languages, currencies (if any fee/payments), and legal frameworks.

Benefits

- Broader adoption potential.
- Flexibility for diaspora or multinational elections.

Implementation notes

- Architect UI to support i18n (gettext/message catalogs or ICU MessageFormat).
- Build locale-aware date/time and number handling.
- Externalize all strings and legal text to translation files.
- Pluggable KYC adapters to support local identity providers.

Security & privacy

- Localize data residency: deploy regionally to meet data localization laws.
- Different jurisdictions will have different PII rules — abstract data handling layers to change retention/destruction policies per region.

Regulatory & legal

- Assess local electoral laws, data protection laws (GDPR, CCPA, etc.).
- Implement region-specific consent flows and privacy notices.

Dependencies

- Translation resources and QA testers for each language.
- Local legal counsel.

Acceptance criteria

- UI fully available in target languages.
- KYC integrations work per country.
- Legal language adapted to local regulations.

Suggested test cases

- Language switch preserves form validation and layouts.
- Regional deployment handles data residency correctly.

5. International Website & Multi-Tenant Architecture

Purpose: host multiple election instances (countries, states, organizations) from a single platform with tenant isolation.

Benefits

- Reusable platform for different clients / jurisdictions.
- Easier upgrades and centralized maintenance.

Implementation notes

- Implement multi-tenant separation at DB and application layer (schema per tenant or tenant ID scoping).
- Tenants should have isolated admin consoles, branding, and policy settings.
- Provide self-service tenant provisioning with a whitelist and approval workflow.

Security & privacy

- Strong isolation to prevent data leakage across tenants.
- Tenant-level Access Control and encrypted backups per tenant.

Dependencies

- Scalable cloud infra, identity provider that supports tenant scoping.

Acceptance criteria

- No cross-tenant data access.
- Admin can manage tenant-specific settings and content.

Suggested test cases

- Attempt cross-tenant data queries (should fail).
- Tenant creation and deletion flows.

6. Offline Voting & Low-Connectivity Modes

Purpose: enable voting in low or intermittent network environments.

Benefits

- Accessibility in remote areas.
- Increased participation.

Implementation notes

- Implement signed, offline ballots that sync when connectivity is available.
- Use secure local storage with tamper-detection, and require periodic re-synchronization with a trusted node.
- Provide physical kiosk mode with hardware security and one-time sync tokens.

Security & privacy

- Offline ballots must be encrypted and signed.
- Sync process must detect duplicates and replay attacks.

Dependencies

- Digital attestation for kiosks, rugged devices.

Acceptance criteria

- Offline ballot accepted and securely synchronized later.
- No duplication or ordering problems.

Suggested test cases

- Long offline period then sync.
- Interrupted sync scenarios.

7. Mobile Applications (iOS and Android) & Progressive Web App

Purpose: provide native mobile apps and/or PWA for better UX and offline capabilities.

Benefits

- Easier authentication via platform biometrics.
- Push notifications for election events.

Implementation notes

- Use native frameworks or cross-platform (React Native / Flutter).
- Implement secure key storage (Android Keystore, iOS Keychain).
- PWA for light-weight access on low-end devices.

Security & privacy

- Secure local storage and secure updates.
- Harden app against reverse engineering (obfuscation, tamper detection) while avoiding reliance on obscurity.

8. Integration of Real Blockchain Networks and Smart Contracts

Purpose:

The current VoteChain implementation uses a **simulated or centralized ledger** only for demonstration purposes. It does **not** utilize a real decentralized blockchain network such as Ethereum, Hyperledger Fabric, Polygon, Solana, or any on-chain smart contract platform.

A key future enhancement is to integrate **actual blockchain technology** and execute all critical election operations through **audited smart contracts**.

Benefits

- **True decentralization** — no single party can alter votes.
- **Transparent and tamper-evident** auditability using public blockchain explorers.
- **Smart contract-based automation** (vote recording, eligibility checks, tallying).
- **Higher trust** from stakeholders, researchers, and auditors.
- **Interoperability** with existing blockchain ecosystems and Layer-2 networks.

Implementation Notes

- Choose a blockchain suitable for e-voting (e.g., **Ethereum, Polygon, Hyperledger Fabric, Avalanche, Algorand**).
- Develop **smart contracts** for:
 - voter registration and verification hash storage
 - vote casting (encrypted or committed ballots)
 - tally computation triggers
 - event logging and audit trails
- Use **Layer-2 scaling solutions** or sidechains to reduce gas cost and allow high throughput.
- Evaluate **off-chain/on-chain hybrid architecture** for privacy (e.g., store ballot commitments on-chain but vote content off-chain).
- Integrate wallets or secure key-storage systems for voters.
(MetaMask, WalletConnect, mobile secure enclaves, hardware wallets, etc.)

Security & Privacy Considerations

- **Never store raw voter identities or actual vote contents on the blockchain.**
Use cryptographic commitments or zero-knowledge techniques.
- Smart contracts must undergo:
 - formal security audits
 - fuzz testing
 - static analysis
 - testnet trials
- Managed key custody or delegated wallet models should be evaluated for non-technical users.

Regulatory & Legal

- Public blockchains introduce data residency, sovereignty and privacy concerns.

Compliance with national election laws and data protection policies must be ensured.

- Evaluate whether permissioned blockchain networks are required for governmental elections.

Acceptance Criteria

- All vote-related transactions must be recorded on a real blockchain.
- Smart contracts must pass audits and remain immutable after deployment.
- The system should allow independent third parties to verify ballots and tallies on-chain.